

Your AI agent never
reads your code

— your repo doesn't fit in the window —

**So it indexes and queries it,
like an IDE. Not a chatbot.**

watch it navigate →

1 READS ANY LANGUAGE

One parser, three languages: it outlines Go, Rust and JavaScript without a compiler for any of them.

```
tool: outline
{"path":"server.go"}
```

```
result: outline
# Outline of server.go
type Server struct { ... }
func (s *Server) Start() error
func NewServer(port int) *Server
```

```
tool: outline
{"path":"lib.rs"}
```

```
result: outline
# Outline of lib.rs
struct Point { ... }
fn distance(a: Point, b: Point) -> f64
```

```
tool: outline
{"path":"widget.js"}
```

```
result: outline
# Outline of widget.js
class Widget { ... }
  render()
function makeWidget(kind)
```

2 KNOWS WHERE THINGS LIVE

“Where is Order defined?” — it queries a symbol index built over the repo and answers with the file and line.

```
tool: find_symbol  
{"name":"Order"}
```

```
result: find_symbol  
1 definition(s) for 'Order':  
models.py:9: class Order
```

```
tool: finish  
{"summary":"Order class is defined in models.py at line 9."}
```

```
result: finish  
Order class is defined in models.py at line 9.
```

```
state  
evaluating
```

```
evaluation  
No tests found; sources compile cleanly
```

```
summary  
Order class is defined in models.py at line 9.
```

```
telemetry  
3 model calls · 5648 tokens · 6.5s · $0.00
```

3 KNOWS WHAT BREAKS

“I'm changing User.” It names api.py and db.py — and skips billing.py, which imports a different class from the same file.

```
state  
executing
```

```
tool: find_importers  
{"name": "User"}
```

```
result: find_importers  
2 importer(s) of 'User':  
api.py:1: imports models.User  
db.py:1: imports models.User
```

```
tool: finish  
{"summary": "Importer findings: api.py and db.py import models.User."}
```

```
result: finish  
Importer findings: api.py and db.py import models.User.
```

```
state  
evaluating
```

```
evaluation  
No tests found; sources compile cleanly
```

4 SEARCHES EVERYTHING

One search, Python and Go together — every TODO in the project, with file and line.

```
tool: search_text  
{"query": "TODO"}
```

```
result: search_text  
3 match(es) for 'TODO':  
billing.py:3: # TODO: handle refunds  
db.py:3: # TODO: move the connection string into config  
server.go:8: // TODO: graceful shutdown
```

```
step 2  
{"name": "finish", "arguments": {"summary": "Found TODOs in billing.py:3, db.py:3, server.go:8."}}
```

```
tool: finish  
{"summary": "Found TODOs in billing.py:3, db.py:3, server.go:8."}
```

```
result: finish  
Found TODOs in billing.py:3, db.py:3, server.go:8.
```

```
state  
evaluating
```

```
evaluation  
No tests found; sources compile cleanly
```

```
summary  
Found TODOs in billing.py:3, db.py:3, server.go:8.
```

```
telemetry  
3 model calls · 5661 tokens · 3.7s · $0.00
```

Part 4 — earlier: “8 things a local AI coding agent can do” · “I gave an AI agent shell access” · “What makes an AI coding agent production-grade?”

Want to build one?

This entire agent is a book —
and a free, open-source repo.



Get the book (Kindle)

amazon.com/dp/B0H8B6QXXX · amazon.in/dp/B0H8B6QXXX



Clone the code (MIT)

github.com/Natarajan-R/local-ai-coding-agent

Follow for the build, one file at a time.

Natarajan Ramasamy